RISKORA

AI assisted micro lending risk assessment and decision support

Review 2 project report

Software Design and Development Project

Institution _____

School or department _____

Course code and class ID _____

Academic year and semester _____

Student name and registration number _____

_____

Guide name and designation _____

Review date _____

Guide signature _____

Internal examiner 1 _____

Internal examiner 2 _____

Technical status and verification date 10 October 2026

Abstract and contents

RISKORA is a local decision-support prototype for reviewing micro-lending requests. It combines borrower validation, calibrated default prediction, funding concentration analysis, repayment schedules and a browser report. A shared schema maps borrower fields consistently across the API, SQLite database and inference service. The browser uses HTML, CSS and JavaScript; a Python Tornado server supplies JSON APIs. A scikit-learn HistGradientBoosting classifier with sigmoid calibration estimates default probability.

The model is trained on 12,000 synthetic records, split into 8,400 training, 1,800 validation and 1,800 test records. Stored test results show ROC-AUC 0.7919, average precision 0.4023 and Brier score 0.0821. At a threshold of 0.20, precision is 36.2% and recall is 40.2%. These results demonstrate an experimental pipeline, while missed defaults and synthetic data limit claims about actual lending performance.

Review 2 work includes reproducible setup, startup checks, validated loan creation, restored case details, exact allocations, duplicate-payment guards, correct paid-principal exposure and responsive navigation. All 50 automated tests passed on 10 October 2026. Authentication, enforced approval stages, partial payments and real-data validation remain incomplete. The application does not transfer funds or approve real borrowers.

Keywords: credit risk, micro lending, probability calibration, synthetic data, decision support, repayment allocation

Section	Page
Introduction and problem statement	3
Objectives scope and proposed system	4
Literature survey and findings	5-6
Methodology and model pipeline	7-8
Software requirements	9
System architecture	10
UML class use case and sequence diagrams	11-13
Database and API design	14
Implementation and project plan	15
Sample code and reproducible setup	16
Sample screenshots	17-18
Results and evaluation	19-20
Limitations future work and summary	21
References	22-23

1 Introduction and problem statement

A loan reviewer needs to connect borrower information, credit risk, the requested amount and repayment terms. When these values move between forms, model inputs and stored records, inconsistent field names or stale displays can change the apparent result. A risk score alone does not show whether the data are complete, which lender carries the exposure, or how an actual payment changes the balance.

RISKORA addresses this software integration problem through a shared input contract and a case-based browser workflow. The reviewer can inspect a saved request, run an assessment, change hypothetical inputs, compare funding structures and examine repayment and exposure information. An academic synthetic dataset allows implementation tests without collecting real borrower records.

1 1 Problem statement

Develop a reproducible micro-lending decision-support application that preserves borrower values across entry points, estimates default probability, exposes data-quality problems and keeps financial displays consistent with stored payments. The application must distinguish borrower risk from lender concentration and separate hypothetical previews from saved commitments.

1 2 Existing approach and its limits

A spreadsheet or isolated scoring script can calculate individual values, but it does not by itself provide a validated API contract, persistent case history, controlled previews or reliable restoration of a repayment view. These are design limitations of a disconnected workflow, rather than a claim that all existing lending platforms lack such controls.

1 3 Project contribution

The contribution is an integrated prototype with regression evidence. Its main improvements are consistent normalization, startup validation, dependable case restoration and correct payment-based exposure. Predictive performance is reported separately from software correctness. Passing application tests does not establish model fairness, real-world accuracy or lending suitability.

1 4 Human and AI collaboration

The intended relationship is augmentation: the model supplies a probability estimate and the reviewer examines evidence. Scenario changes support sensitivity analysis, not causal claims. No LLM chatbot, autonomous agent or document-understanding system is implemented. Although the repository name includes Agentic AI, the current code is a conventional ML and rules application.

2 Objectives scope and proposed system

Objective	Review 2 evidence
Consistent borrower handling	Shared aliases, validation ranges and category normalization.
Reproducible setup	Pinned requirements, configuration and three-artifact startup checks.
Useful risk assessment	Calibrated PD, risk bands, quality warnings and policy indicators.
Reliable reviewer workflow	Queue filters, report rendering, case restoration and mobile navigation.
Correct financial displays	Exact cent allocation, duplicate guards and paid-principal exposure.
Repeatable verification	50 isolated unit and API tests; Windows and Linux CI configuration.

2 1 Implemented scope

The server supports loan creation, listing, detail retrieval, saved assessments, scenario simulation, funding calculation and saving, repayment generation, full-installment recording, portfolio summaries, dataset browsing and model metrics. The frontend works with seeded or saved requests. A dedicated browser loan-creation form is still required.

2 2 Proposed system

The browser calls a same-origin Tornado API. Borrower inputs pass through a canonical schema before quality rules and inference. The model and preprocessor are loaded from validated artifacts. SQLite stores borrower and loan records together with assessments, funding positions, schedules, payments and audit events. The browser assembles the report from active and restored case data.

2 3 Intended complete workflow

Reviewer login → Create loan → Validate borrower data → Assess risk → Human review → Commit funding → Generate schedule → Record payments → Update exposure → Produce report.

This is the target workflow. The current reviewer-name screen is a label, not secure login, and human approval transitions are not fully enforced. Saving a funding structure is a local database operation. Production lending, KYC verification, payment rails, automatic approvals and regulatory compliance are outside the demonstrated scope.

3 Literature survey

Eight studies were selected for credit-scoring evaluation, probabilistic prediction, model explanation and alternative-data considerations. Their findings inform the design; they do not validate RISKORA on real loans. References [1]-[8] provide publication links.

3 1 Lessmann and colleagues 2015 [1]

The credit-scoring benchmark compares classification approaches in retail credit data. Its value is systematic evaluation rather than assuming a single algorithm is best. Results depend on datasets and measures. RISKORA compares logistic regression, random forest and gradient boosting, but still needs real-data and repeated-split evaluation.

3 2 Breiman 2001 [2]

Random forests combine randomized tree predictors. They provide a useful nonlinear baseline with less dependence on a single tree. Ensemble complexity makes direct interpretation harder. RISKORA includes random forest in its comparison; its higher validation ROC-AUC must not be hidden when explaining the selected model.

3 3 Friedman 2001 [3]

Gradient boosting fits additive models through stagewise optimization. It provides a basis for learning nonlinear patterns using successive predictors. Training choices and noise affect performance. RISKORA uses histogram-based gradient boosting, an implementation of the broader boosting approach, with separate probability calibration.

3 4 Niculescu Mizil and Caruana 2005 [4]

This study examines probability distortions across classifiers and compares sigmoid and isotonic calibration. It shows why ranking accuracy alone is insufficient when probabilities guide decisions. Calibration quality depends on data and model. RISKORA reports Brier score alongside ROC-AUC and average precision.

3 5 Relevance to the project

The common design lesson is to compare discrimination and probability quality. RISKORA should justify model choice through a declared objective and repeatable selection procedure. Its training script currently chooses HistGradientBoosting explicitly; that is not an automatic search for the highest ROC-AUC.

3 Literature survey continued

3 6 Guo and colleagues 2017 [5]

The paper examines confidence calibration in modern neural networks and evaluates temperature scaling. It supports treating calibration as a separate task. Its architecture-specific results cannot establish calibration quality for a tree-based credit model. RISKORA reports its own held-out metrics rather than borrowing a performance claim.

3 7 Ribeiro Singh and Guestrin 2016 [6]

LIME explains individual predictions through local surrogate models. Its strength is a model-agnostic interface; a local approximation is not a causal account or a complete description of the original model. RISKORA does not implement LIME. Its fixed rule points must remain labeled as policy indicators.

3 8 Lundberg and Lee 2017 [7]

SHAP provides an additive framework that assigns feature contributions to a particular prediction. This is relevant when explanations must describe model behavior. Explanations depend on formulation and reference distribution, so they do not establish causality. RISKORA does not implement SHAP; model-specific explanation is future work.

3 9 Berg and colleagues 2020 [8]

The study evaluates digital footprints for consumer-default prediction and finds complementary information to bureau scores in its setting. It highlights alternative data, while transfer to another population requires validation and careful handling of privacy and proxies. RISKORA uses structured synthetic features and does not collect digital footprints.

3 10 Findings and remaining gap

The literature supports benchmarking, probability evaluation and honest explanation labels. This prototype addresses the integration of these outputs with case persistence and financial displays. It does not claim a new learning algorithm or improved real-world default outcomes. Real-data evaluation, subgroup analysis and reviewer studies are still needed.

4 Methodology and dataset

Development followed a correctness-first process: inspect the repository, identify confirmed defects, normalize borrower inputs, repair saved-state contracts, add isolated regression tests and verify the browser workflow. These repairs establish stable behavior for later features.

4 1 Dataset provenance

The bundled CSV is generated by ml/generate_data.py with seed 42. It contains 12,000 synthetic rows and 18 columns: LoanID, Default and 16 raw predictors. The generator sets borrower distributions and risk relationships programmatically. It follows a common loan-default column schema; the file is not a verified Kaggle download or a sample from an actual lending institution.

Group	Fields
Numeric predictors	Age, Income, LoanAmount, CreditScore, MonthsEmployed, NumCreditLines, InterestRate, LoanTerm, DTIRatio
Categorical predictors	Education, EmploymentType, MaritalStatus, HasMortgage, HasDependents, LoanPurpose, HasCoSigner
Excluded from model input	LoanID identifies a row. Default is the binary training target.

4 2 Normalization and validation

Canonical training names, API aliases and database names map to the same fields. CreditScore, credit_score and creditScore identify the same value. Conflicting aliases, non-finite numbers, invalid categories and noninteger values in integer fields are rejected. Loan creation requires a name and eight numeric fields. Optional omissions remain visible in stored data and quality results.

4 3 Quality and inference

Quality rules assess completeness, validity and contradictions. They do not verify salary documents or identity. Inference can substitute documented defaults for incomplete existing or sandbox profiles, so such predictions are provisional. The quality response must be read with the risk estimate.

4 Model training and inference pipeline

Stage	Implementation
Target isolation	Default becomes y. LoanID and Default are removed from X.
Feature engineering	LoanToIncome and MonthlyBurdenRatio extend 16 raw predictors to 18 inputs.
Stratified split	70% training, 15% validation, 15% test, random_state 42.
Preprocessing	StandardScaler for 11 numeric fields; one-hot encoding for seven categories. 26 transformed features.
Comparison	Logistic regression, random forest and HistGradientBoosting on validation data.
Selected model	HistGradientBoosting selected explicitly in the script.
Calibration	CalibratedClassifierCV, sigmoid method, five folds on the training partition.
Export and startup	Model, preprocessor and metrics saved and checked before service startup.

4 4 Feature calculations

LoanToIncome = LoanAmount / max(Income, 1). MonthlyBurdenRatio = (LoanAmount / max(LoanTerm, 1)) / max(Income / 12, 1). The latter is a principal-only installment-to-income proxy, not an interest-inclusive EMI affordability measure.

4 5 Risk bands and score

Low risk means PD below 0.08. Medium risk means 0.08 ≤ PD < 0.20. High risk means PD ≥ 0.20. The display score is a piecewise transformation of PD, bounded from 5 to 99. It is not a bureau score or a second trained prediction. A score of 96 therefore does not mean a 96% probability of default.

4 6 Methodological limitations

The preprocessor is fitted on the outer training partition before calibration folds are formed. Stricter fold isolation would put preprocessing inside the estimator pipeline. The 0.20 threshold has a stated 4:1 cost assumption but no demonstrated cost-minimization study. The coefficient list in metrics comes from the logistic-regression baseline, not from the selected boosting model.

5 Software requirements

Functional requirement	Current status
Create and validate loan	Implemented through POST /api/loans; browser creation form pending.
Search and filter requests	Implemented, including analyzed and verification filters.
Assess and simulate risk	Implemented; simulations do not save hypothetical changes.
Preview and save funding	Implemented; dropdown changes produce a preview.
Generate repayment schedule	Explicit action; monthly, bi-monthly and flexible modes.
Record and restore payments	Exact full-installment payment; duplicates rejected; partial payment pending.
Calculate exposure and report	Paid principal deducted; report refreshes on navigation.
Authentication and approval	No secure login or complete enforced approval state machine.

5 1 Nonfunctional requirements

Reproducibility is supported by pinned dependencies and configuration paths. Reliability is checked through temporary-database tests and saved-state regressions. Maintainability is supported by separate schema, inference, funding and repayment modules. Mobile navigation and accessibility state were repaired. There is no measured latency target, load-test result or multi-user capacity claim.

5 2 Runtime and development tools

The verified runtime is CPython 3.14 with scikit-learn 1.9.1, NumPy 2.5.2, pandas 3.0.5 and Tornado 6.5.10. SQLite supplies local storage. JavaScript, HTML and CSS run in a browser. Joblib stores trained artifacts. Git and GitHub hold source history; CI is configured for Windows and Linux. No GPU is required by the implemented tree-based model.

5 3 Security requirements

Static delivery is restricted to allowlisted frontend assets, and the server defaults to loopback. These controls do not authenticate API callers. Credentials, sessions, roles, authorization, controlled origins and trusted reviewer audit identities must be completed before shared or public use.

6 System architecture

The application separates browser presentation, request handling, analytical services and local persistence. Artifact and dataset paths come from backend/config.py. Inference is synchronous within the request handler; Tornado support does not by itself provide a separate ML worker pool.

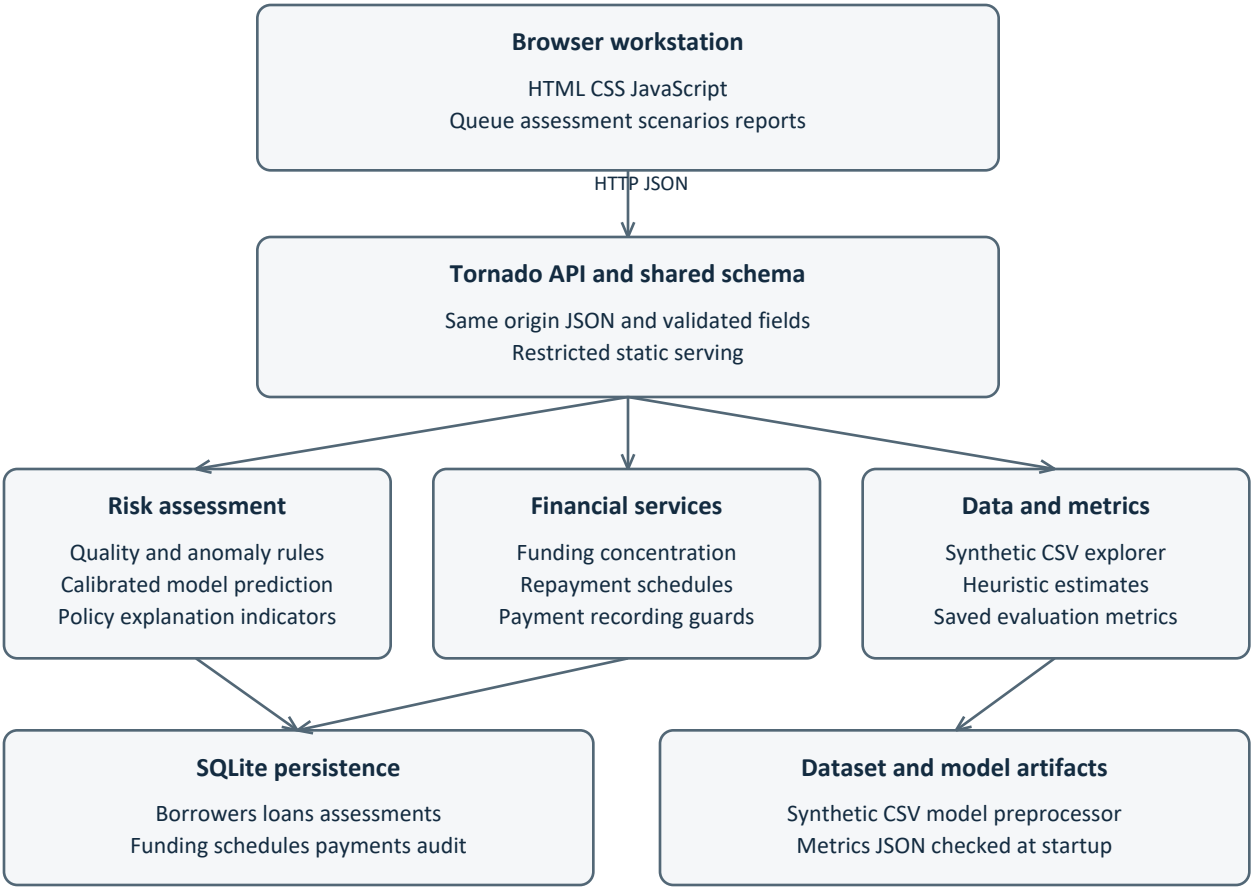


Figure 1 Current RISKORA component architecture

6 1 Request and persistence boundaries

Loan detail responses restore the latest assessment, funding and repayment records. The response adds lender summaries and term fields needed by the frontend, so refresh uses the same display contract as a newly generated schedule. Latest-record queries use timestamp and ID ordering to resolve ties.

6 2 Offline and runtime separation

Generation and training are explicit offline commands. Startup validates existing files and fails when required artifacts are missing or incompatible. It does not retrain automatically. The explorer reads the synthetic CSV and applies a heuristic estimate; it is separate from calibrated borrower inference.

7 UML class diagram

The class diagram shows implemented request handlers and service dependencies. Handler inheritance reuses JSON parsing and response behavior. Service calls perform analysis and calculations; SQLite operations remain in the server and database module.

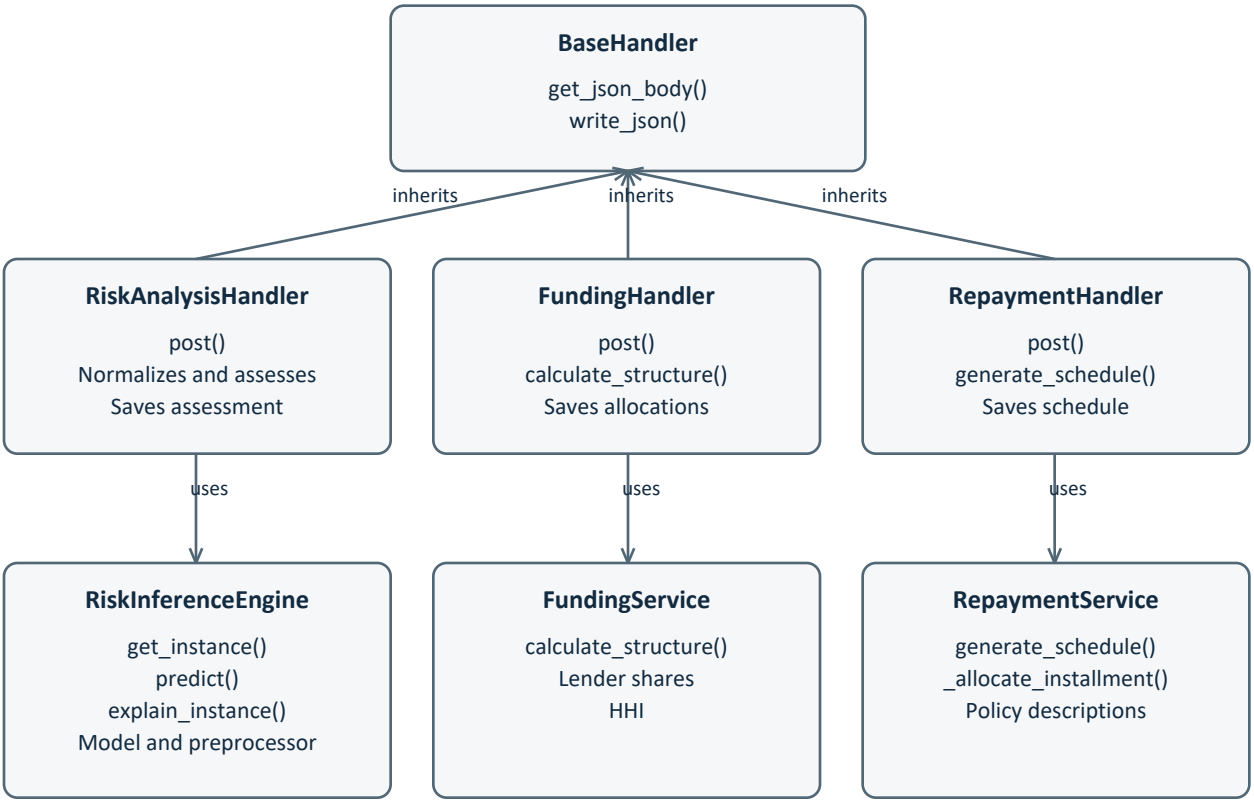


Figure 2 Selected implemented classes and their relationships

7 1 Responsibilities

RiskAnalysisHandler combines normalized borrower values, quality and anomaly rules and RiskInferenceEngine output before saving an assessment. FundingHandler calls FundingService and saves a structure when a loan ID is supplied. RepaymentHandler calls RepaymentService and persists generated installments.

RecordPaymentHandler checks that a schedule exists, locates the installment, validates the amount and rejects an already-paid installment. It updates schedule status and inserts payment and audit events in the transaction. The diagram focuses on the main calculation path rather than every endpoint class.

7 2 Design distinction

Borrower, Loan and Payment are database entities, not implemented Python domain classes. Their relationships are described in the database section. The browser controller stores activeCase, activeAnalysis, activeFunding, fundingPreview and activeRepayment as frontend state.

7 UML use case diagram

The reviewer is the primary actor for the browser demonstration. The boundary contains investigation, scenario, financial-display and reporting actions. Secure authentication and formal approval authorization are future use cases, excluded from this implemented diagram.

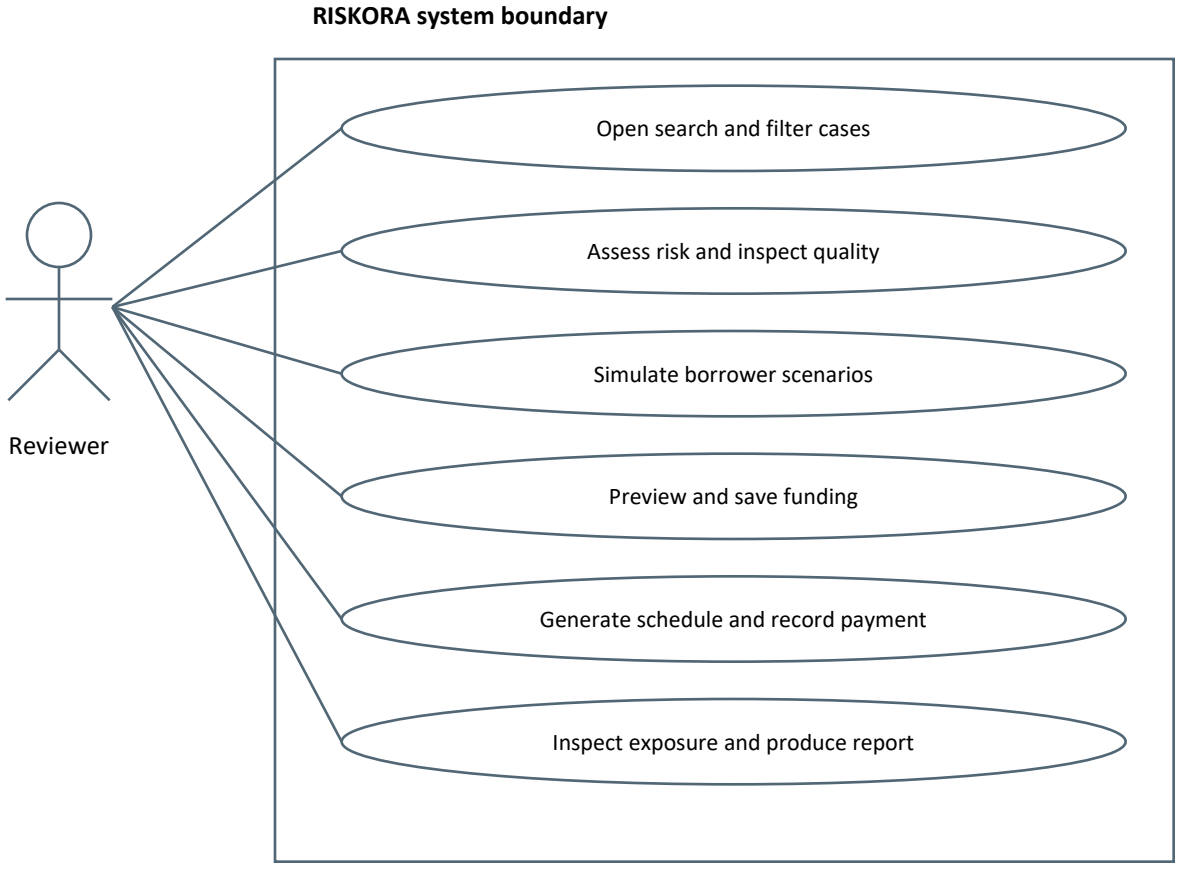


Figure 3 Implemented reviewer use cases

Use case	Precondition and outcome
Assess risk	Saved case selected; prediction and quality results returned and saved.
Preview funding	Mode and lender count selected; preview displayed without persistence.
Record payment	Existing schedule and exact unpaid installment amount; status and events saved.
Produce report	Assessment available; browser assembles current case data for viewing and printing.

An API client can also create a validated loan, but no dedicated browser form is implemented. No external bank, bureau or payment processor is an actor in the current system.

7 UML sequence diagram

This sequence illustrates a saved-case assessment. Canonical mapping happens before analytical services use borrower data. The result renders in the browser, and later detail retrieval restores stored information.

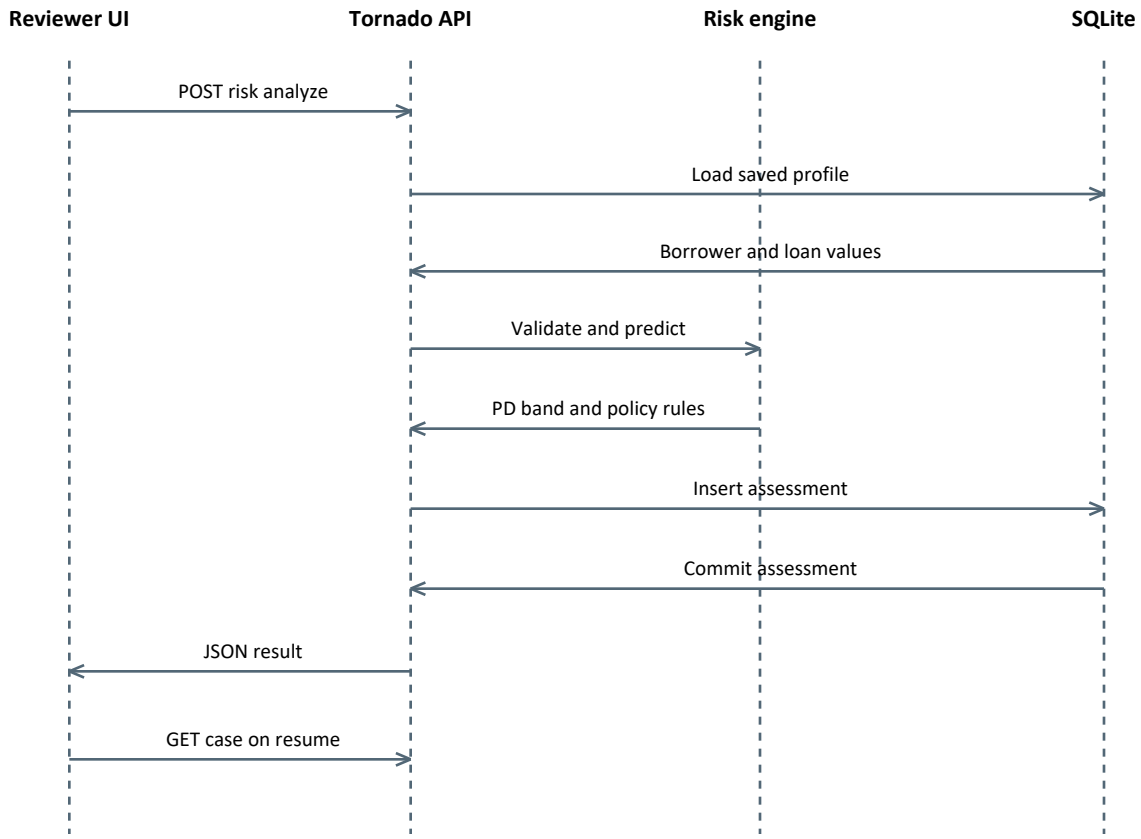


Figure 4 Saved loan assessment sequence

7 3 Scenario and payment alternatives

A scenario request loads the baseline case, applies normalized hypothetical overrides and returns changes in PD and score. It does not insert a loan or overwrite baseline fields. Resetting exact values must produce zero difference, which is covered by regression tests.

A payment request validates the existing schedule and installment, rejects invalid or duplicate payment, updates status, inserts payment allocations and writes an audit event. The browser reloads the saved case and recalculates exposure from paid principal. Reconfiguration is rejected once payment records exist.

8 Database and API design

Table	Purpose and relationship
users	Reviewer labels and demo login records; no secure authentication guarantee.
borrowers	Identity label, income and demographic or employment fields.
loans	Borrower reference, amount, credit profile, rate, term and status.
risk_assessments	Loan reference, model version, PD, band, score and quality or rule JSON.
funding_positions	Loan reference, mode, HHI and lender positions JSON.
repayment_schedules	Loan reference, terms, totals and installments JSON.
payments	Loan reference, installment number, amount and allocations JSON.
audit_logs	Event type, entity ID, reviewer text and event details.

One borrower may have multiple loans. A loan may have multiple assessments, funding structures, schedules and payment events. The application restores the latest structure or schedule. Payments identify loan and installment without a dedicated schedule-version foreign key; this limits safe future rescheduling and migration design.

8 1 Main API groups

API route	Role
GET and POST /api/loans	List or create validated loans.
GET /api/loans/{id}	Restore case and assessment, funding and repayment details.
POST /api/risk/analyze and /simulate	Save assessment or calculate unsaved scenario.
POST /api/funding/optimize	Compute preview; save when loan ID provided.
POST /api/repayment/generate and /record	Save schedule or record full installment.
GET /api/exposure/portfolio	Portfolio summary and recent funding.
GET /api/dataset/explorer and /model/metrics	Dataset rows and saved metrics.

9 Implementation and project plan

Implementation repaired behavior across the backend, frontend and integration contract. The work below is confirmed by source and tests, rather than a percentage estimate of overall completion.

Phase	Work and current status
Setup and configuration	Completed: pinned dependencies, configurable paths, explicit startup checks and isolated tests.
Borrower handling	Completed: loan creation, shared aliases, validation and corrected queue filtering.
Frontend integration	Completed repairs: report navigation, saved terms and summaries, empty-case reset, exact scenario values, mobile navigation and aria-hidden state.
Funding and repayment	Completed repairs: preview separation, explicit schedule actions, duplicate guards, exact allocation and paid-principal exposure.
Trustworthy ML reporting	Partial: synthetic and heuristic labels; model-specific explanation and stronger selection evaluation pending.
Security and complete workflow	Pending: real authentication, authorization, approval state machine and trusted audit identities.
Deployment and maintenance	Pending: migrations, backup and restore, production deployment and operational tests.

9 1 Defects resolved by Review 2

Reports could open blank, recorded payments could replace fields with undefined values, and outstanding principal could decrease before payment. A slider rounded credit score 548 to 550, creating a nonzero scenario difference. Mobile navigation disappeared and the visible application remained aria-hidden. Repairs now preserve API display fields and remove stale state when changing cases.

9 2 Remaining integration concerns

The portfolio endpoint supplies summaries; active-case exposure performs its own paid-allocation calculation. A consolidated ledger service should make the database authoritative for all balances. Some older recommendation and metrics-screen wording still implies stronger verification or model attribution than the implementation supports.

9 Sample code and reproducible setup

9 3 Shared field mapping

```
ALIASES = {
    "LoanAmount": ("LoanAmount", "loan_amount",
                   "loanAmount", "amount"),
    "CreditScore": ("CreditScore", "credit_score",
                    "creditScore", "credit"),
}
```

This excerpt from backend/schema.py gives API and saved-field aliases a canonical destination. Validation checks type, range and category before downstream services receive the profile. Conflicting aliases are rejected.

9 4 Payment based outstanding principal

```
outstanding = Math.max(0,
  activeRepayment.original_principal -
  activeRepayment.installments
    .filter(i => i.status === "PAID")
    .reduce((sum, i) => sum + i.principal_component, 0)
);
```

This equivalent frontend excerpt subtracts principal from paid installments, rather than reading the balance after the next unpaid installment. Lender balances subtract each lender's paid principal allocations, including waterfall allocations.

9 5 Local setup and verification

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe run.py --check
.\.venv\Scripts\python.exe run.py
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
```

Open http://127.0.0.1:8000 after startup. Set RISKORA_DB_PATH to a separate demo database when demonstrating writes. Configuration also supports host, port, dataset and artifact directory paths. The reviewer name is a label, not credential-based login.

9 6 Financial calculations

For principal P, periodic rate r and n installments, EMI = $P \times r \times (1 + r)^n / ((1 + r)^n - 1)$. When r is zero, EMI = P / n. Final principal absorbs rounding so the scheduled balance reaches zero. HHI is the sum of squared lender percentage shares; a single lender has HHI 10,000. Principal allocation supports pro rata by remaining balances, largest outstanding lender first, or insertion-order FIFO. Interest is allocated by committed shares, with residual cents distributed so totals reconcile.

9 Sample screenshots of case investigation

Screens captured from the current local application on 10 October 2026 using an isolated demonstration database.

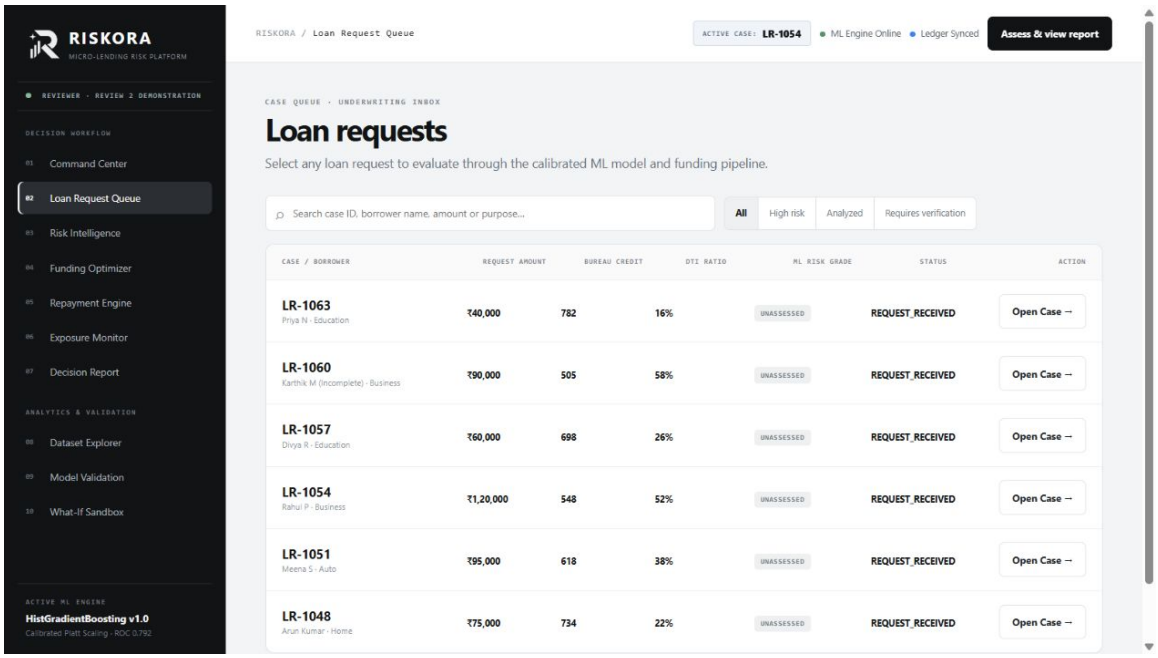


Figure 5 Loan request queue

Saved demo requests show borrower values and available filters.

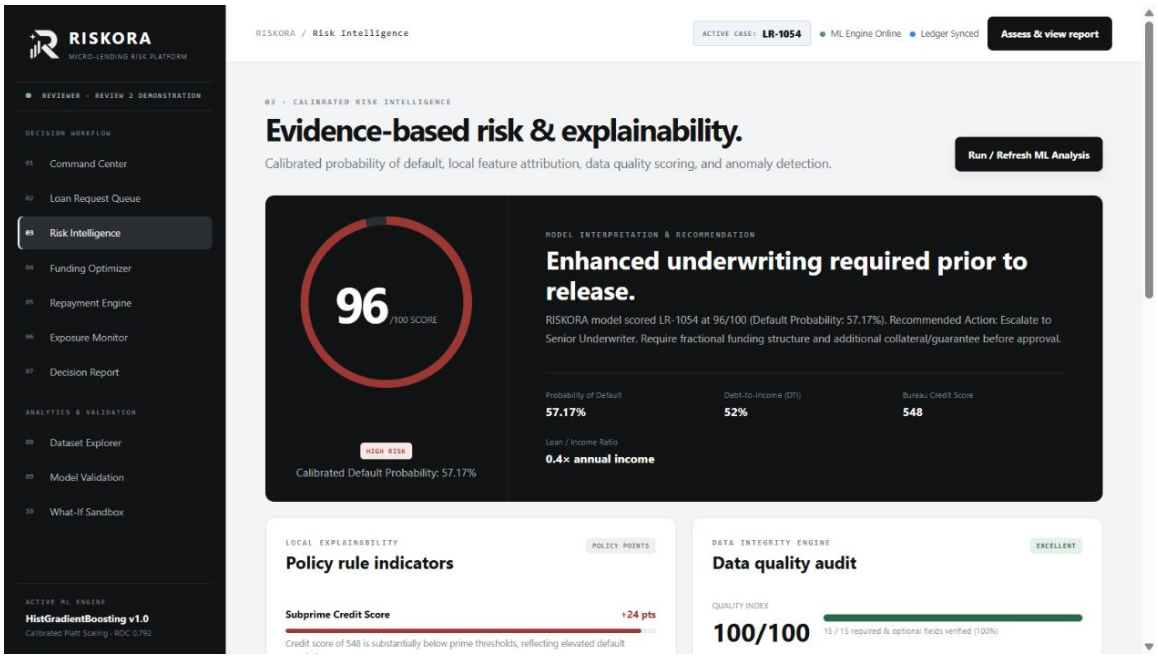


Figure 6 Calibrated risk assessment

LR-1054 returns PD 57.17%, HIGH band and display score 96. Data-quality scoring checks input rules, not documentary verification.

9 Sample screenshots of reporting and metrics

Screens captured from the current local application on 10 October 2026 using an isolated demonstration database.

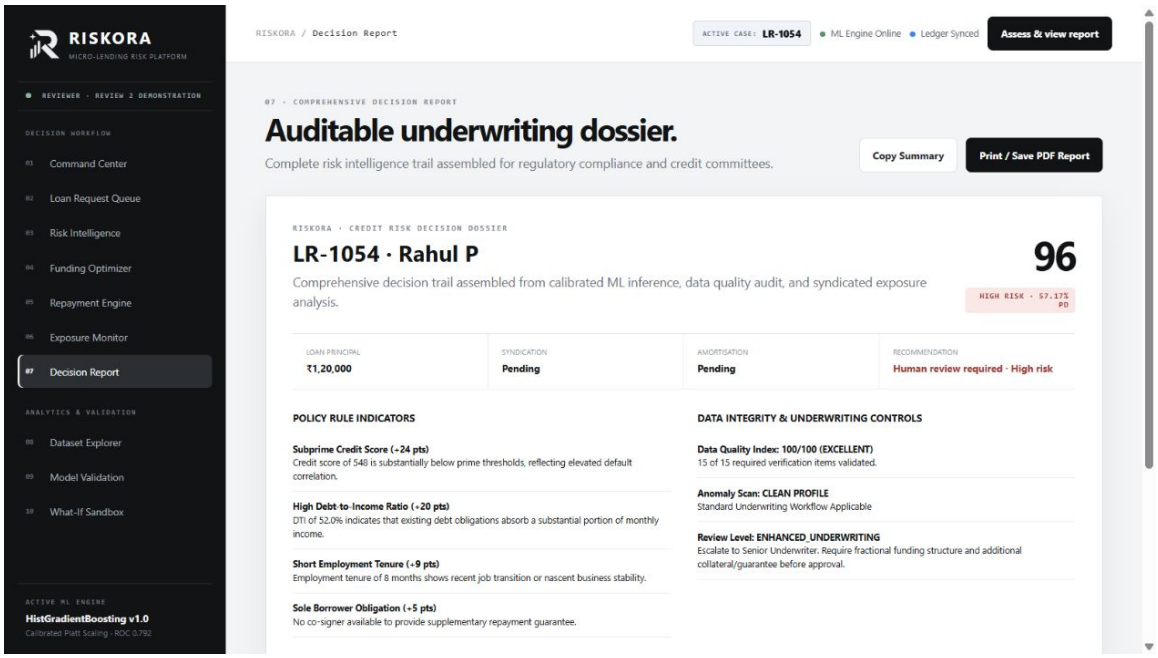


Figure 7 Case decision report

The report renders on navigation and requires human review. Funding and repayment remain pending in this captured case.

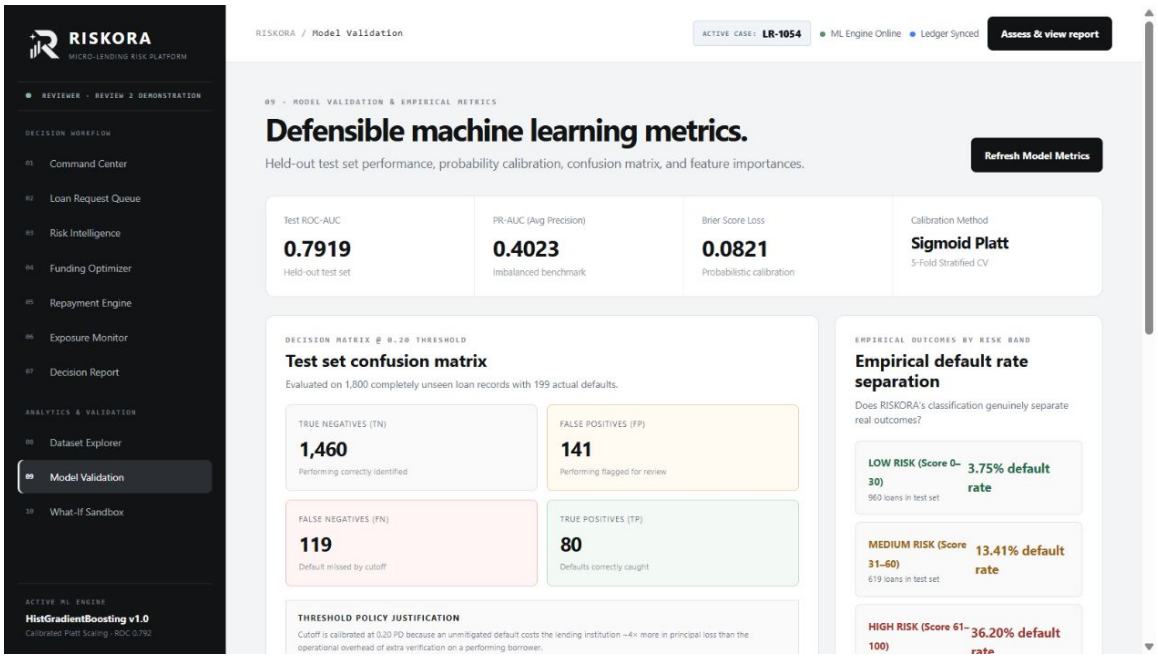


Figure 8 Stored model evaluation

Metrics describe the synthetic held-out test set. Existing wording such as actual defaults or global attribution must be read as synthetic labels and baseline coefficients, not real-loan evidence.

10 Results and model evaluation

Values below are recorded in ml/artifacts/model_metrics.json for the saved run dated 16 September 2026. Training was not repeated for this report. Candidate results use validation data; final calibrated results use the separate test set.

Validation candidate	ROC AUC	Avg precision	Brier	F1
Logistic regression	0.8133	0.4546	0.1687	0.2524
Random forest	0.8164	0.4343	0.1380	0.2670
HistGradientBoosting	0.7994	0.4263	0.0816	0.4169

Random forest has the highest validation ROC-AUC, and logistic regression has the highest average precision. HistGradientBoosting has the lowest Brier score and highest F1 at the comparison threshold. These results support a tradeoff discussion, not a claim that the selected model wins every measure.

Calibrated test measure	Recorded result
Test size	1,800 records; 199 synthetic defaults
ROC AUC	0.7919
Average precision	0.4023
Brier score	0.0821
Precision at PD 0.20	0.3620
Recall at PD 0.20	0.4020
F1 at PD 0.20	0.3810

10 1 Confusion matrix and interpretation

Actual class	Predicted non default	Predicted default
Non default	TN 1,460	FP 141
Default	FN 119	TP 80

Precision = 80 / (80 + 141) = 36.20%. Recall = 80 / (80 + 119) = 40.20%. The cutoff misses 119 of 199 defaults, or 59.80%. Accuracy = (1,460 + 80) / 1,800 = 85.56%, below the 88.94% accuracy of always predicting non-default. Accuracy alone therefore hides imbalance and does not establish deployment readiness.

10 Evaluation curves and regression evidence

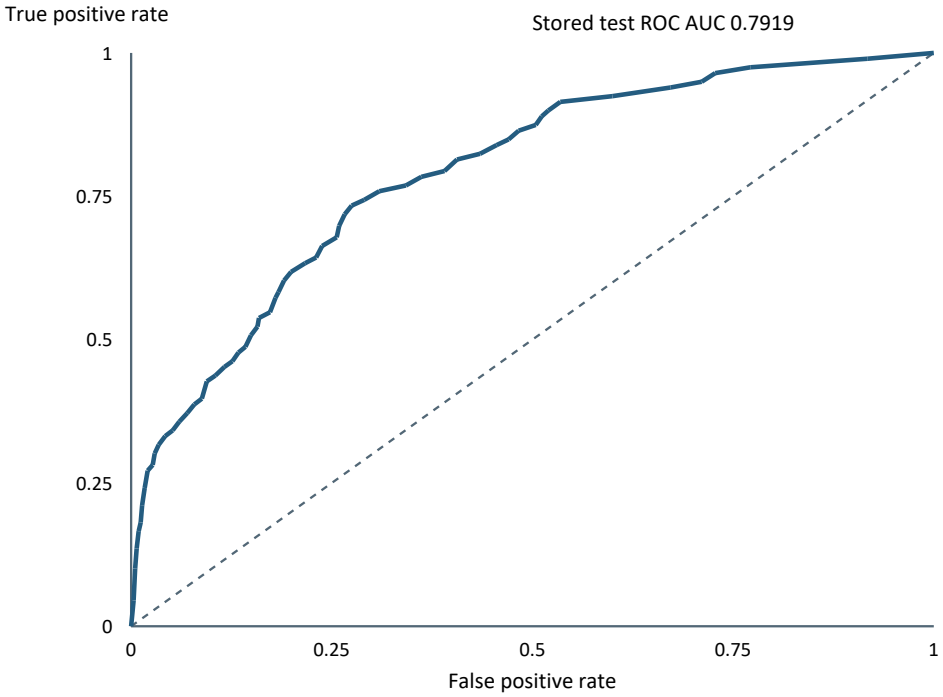


Figure 9 ROC curve reconstructed from stored test points; diagonal shows chance ranking

Risk band	Records	Defaults	Observed rate
Low PD below 0.08	960	36	3.75%
Medium PD 0.08 to below 0.20	619	83	13.41%
High PD at least 0.20	221	80	36.20%

Increasing default rates across bands show separation in this synthetic split. They do not establish calibration in real borrower populations. Brier score should be accompanied by calibration curves and subgroup results.

10 2 Software verification

On 10 October 2026, all 50 tests passed in 23.729 seconds with temporary test databases. Coverage includes loan creation, alias parity, invalid input, latest queue records, verification filtering, unchanged scenarios, static-file restrictions, saved terms and summaries, duplicate and invalid payments, and exact allocation under three policies. CI is configured for Windows and Linux; this report records a local Windows run.

Browser evidence confirms the risk screen and report render for a seeded case. Earlier isolated checks also confirmed repayment restoration, correct pre-payment exposure, clearing unassessed-case displays and mobile navigation. No user-study result, production-load result or measured reduction in review time is claimed.

11 Limitations future work and summary

11 1 Current limitations

Synthetic data reproduce programmed relationships rather than a lending population. There is no specified real-world default observation horizon, independent external validation or subgroup fairness evaluation. Policy points are separate from trained-model explanation. The script hardcodes model choice, and preprocessing should be isolated inside calibration folds.

Reviewer labels do not authenticate users. Endpoints lack a complete approval state machine. Partial payments are rejected rather than accumulated. Payments do not identify schedule versions, audit reviewer text is not a trusted identity, and full assessment input snapshots are not stored. SQLite and synchronous inference have not been load tested for shared deployment.

11 2 Next work in dependency order

Priority	Completion condition
Complete workflow	Borrower form, enforced stages and recorded human approval before funding.
Financial ledger	Partial and idempotent payments, schedule versions and authoritative balances.
Trustworthy ML	Real-data validation, declared selection, calibration curves, subgroup tests and model-specific explanations.
Security and accountability	Credentials, roles, authorization, trusted reviewer IDs and input-version audit.
Deployment and maintenance	Migrations, structured logs, backup and restore and supported deployment checks.

11 3 Summary

RISKORA demonstrates a reproducible ML and rules application connected to a browser workflow and local financial records. Review 2 establishes software correctness through tested normalization, persistence and display repairs. The stored model shows measurable discrimination on synthetic data, but low recall and incomplete security prevent production-readiness claims. Next work should focus on controlled human review, ledger integrity and stronger model validation before agent-assisted features.

References

[1] S. Lessmann, B. Baesens, H.-V. Seow and L. C. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. European Journal of Operational Research, 247(1), 124-136, 2015.

Publication link

[2] L. Breiman. Random Forests. Machine Learning, 45, 5-32, 2001.

Publication link

[3] J. H. Friedman. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189-1232, 2001.

Publication link

[4] A. Niculescu-Mizil and R. Caruana. Predicting Good Probabilities With Supervised Learning. Proceedings of ICML, 2005.

Publication link

[5] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. On Calibration of Modern Neural Networks. Proceedings of Machine Learning Research, 70, 1321-1330, 2017.

Publication link

References [1]-[5] support algorithm comparison and probability evaluation. Publication findings provide background; repository artifacts are the source for RISKORA results.

References continued

[6] M. T. Ribeiro, S. Singh and C. Guestrin. Why Should I Trust You? Explaining the Predictions of Any Classifier. arXiv:1602.04938, 2016.

Publication link

[7] S. M. Lundberg and S.-I. Lee. A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30, 2017.

Publication link

[8] T. Berg, V. Burg, A. Gombovic and M. Puri. On the Rise of FinTechs: Credit Scoring Using Digital Footprints. Review of Financial Studies, 33(7), 2845-2897, 2020.

Publication link

Project evidence and review reference

[9] RISKORA source repository. Architecture, API, schema, training and inference code, repayment service, frontend controller and tests. Snapshot commit 6aabb28. Accessed 10 October 2026.

RISKORA repository

[10] RISKORA model_metrics.json. Saved model version riskora-ml-v1.0, trained 16 September 2026. Source for split counts, candidate comparison, held-out metrics, confusion matrix and risk bands.

[11] Faculty-provided Review 2 reference report, review 2 AI Project.pdf. Used for structure: literature survey, architecture, UML, implementation and screenshots. Its identity details, chatbot topic and performance claims are not RISKORA evidence.

Literature links checked 10 October 2026. Software evidence is distinct from the external literature and faculty sample.